X-Ray Pneumonia Detection

An app that takes an uploaded chest X-ray, preprocesses the image, applies a trained machine learning model to classify it as healthy or non-healthy, and generates a medical report

Authors:

Helton Mendonça fc56870
Jessica Soares fc43356
Duarte Valente fc64335

June 15, 2025

Contents

1 Introduction **2**
- 1.1 Motivation . . . 2
- 1.2 Dataset characterization . . . 2
- 1.3 Business Capabilities . . . 2
- 1.4 Use Cases . . . 2

2 API Specification **3**

3 Requirements **4**
- 3.1 Functional Requirements . . . 4

4 Architecture **5**
- 4.1 Application Architecture . . . 5
  - 4.1.1 Components . . . 5
  - 4.1.2 Interactions between these components . . . 5
  - 4.1.3 Diagram of the Application Architecture . . . 5

5 Implementation **7**

6 Deployment **8**

7 Testing and Evaluation **9**
- 7.1 Stress test . . . 9
- 7.2 Smoke test . . . 9
- 7.3 Takes and Feedback . . . 10
- 7.4 Corrections implemented after feedback from testing . . . 10

8 Cost Analysis **11**
- 8.1 Estimated monthly costs: . . . 11
- 8.2 FinOps Measures . . . 11

9 Conclusions **12**

Chapter 1

Introduction

1.1 Motivation

The aim of this project is to develop an app for AI-assisted diagnosis of Pneumonic Patients that is cloud-native. This would allow doctors to run this app locally on their devices, after it has been trained on the cloud, optimizing cost-efficiency.

1.2 Dataset characterization

The dataset on Pneumonia Chest X-Ray Images was retrieved from Kaggle (`https://www.kaggle.com/datasets/paultimothymooney/chest-xray-pneumonia`). It has 1.24Gb and was updated 7 years ago.

1.3 Business Capabilities

The system provides AI-assisted diagnostic of Pneumonic Patients through chest X-ray analysis.

1.4 Use Cases

1. Helping Doctors to achieve a diagnosis for their patients. By having an AI assistant to have a continuous look on the X-Rays of the patient this could help the doctor adjust his opinion on several other factors and/or symptoms the patient presents to achieve a faster and accurate diagnosis.

2. Acting as a second opinion on previous diagnosis. In some extreme cases where the Doctors could be indecisive of the diagnosis and assuming the tool wasn't used to reach the indecisiveness it could prove to be a second opinion.

3. Ease the understanding of the current situation to the patient and non-specialized individuals. As there might be some difficulty explaining some medical concepts to patients and their families, this tool would present a way to simply explain the X-rays to the untrained eye.

Chapter 2

API Specification

The YAML attachment specifies a REST API using the OpenAPI format to support the use cases.

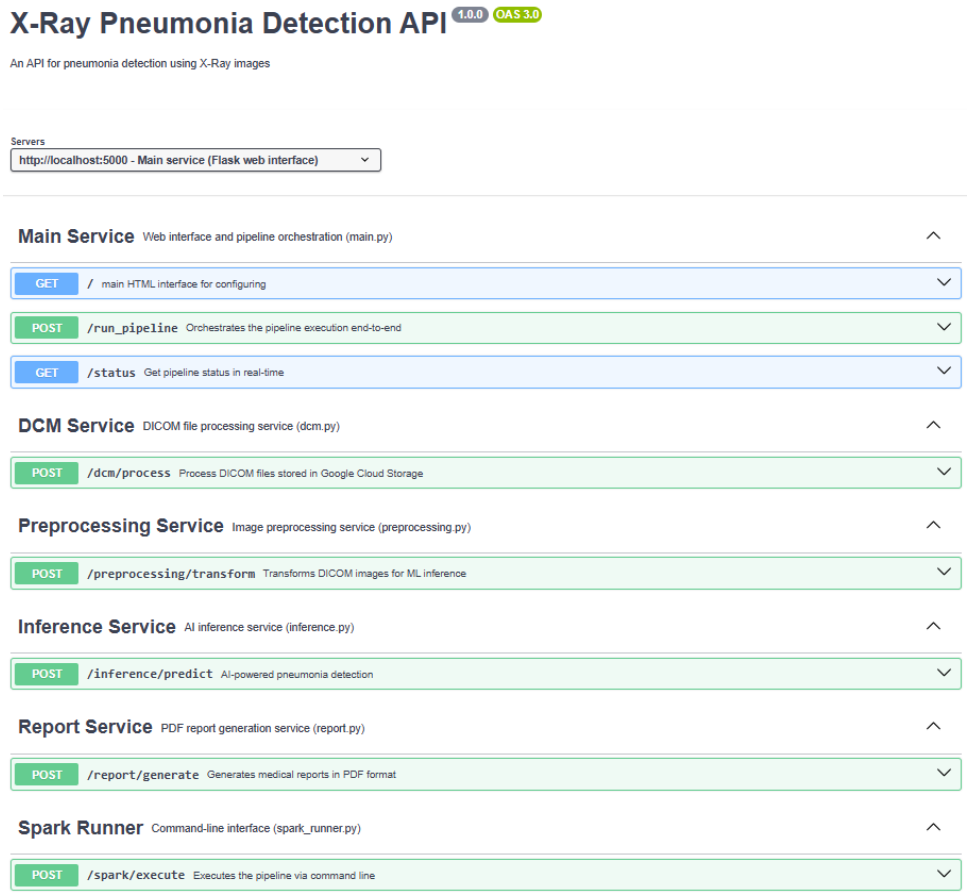


Figure 2.1: API Specification done using `https://editor.swagger.io/`. This YAML allows the code to be automatically generated.

Chapter 3

Requirements

3.1 Functional Requirements

1. Users are able to upload chest X-ray images (DICOM).
2. All uploaded images are resized to a predetermined dimension of 128x128 pixels.
3. This filter reduces some noise through the application of a Gaussian blur.
4. The image contrast is enhanced by extending the pixel's intensity range from the original range to 0 to 255 (histogram equalization).
5. Certain regions of interest within the lungs are detected and cropped using segmentation techniques.
6. Preprocessed images are stored in the cloud.

Chapter 4

Architecture

4.1 Application Architecture

4.1.1 Components

- Image upload component
- Preprocessing component
- Cloud Storage component
- Access Image component

4.1.2 Interactions between these components

- User uploads an image with a REST API (HTTP POST).
- The DCM, Preprocessing, Inference and Report components fetch the image with a REST API framework.
- This image (already processed) is stored in the cloud storage component and the cloud storage be accessed with a REST API.
- The processed image can be accessed with a REST API framework.

4.1.3 Diagram of the Application Architecture

The diagram of the application architecture is represented below. The user uploads the image, an X-ray of a patient. This image is then sent to the DCM service that will separate the data into images and metadata. The images will be sent to the preprocessing service to be preprocessed through several steps: resize, noise reduction, contrast adjustment and image cropping. The image is then sent to the inference service, that will apply the model to the image and classify it as healthy or non-healthy. This information, along with the metadata retrieved initially, are then sent to the report service.

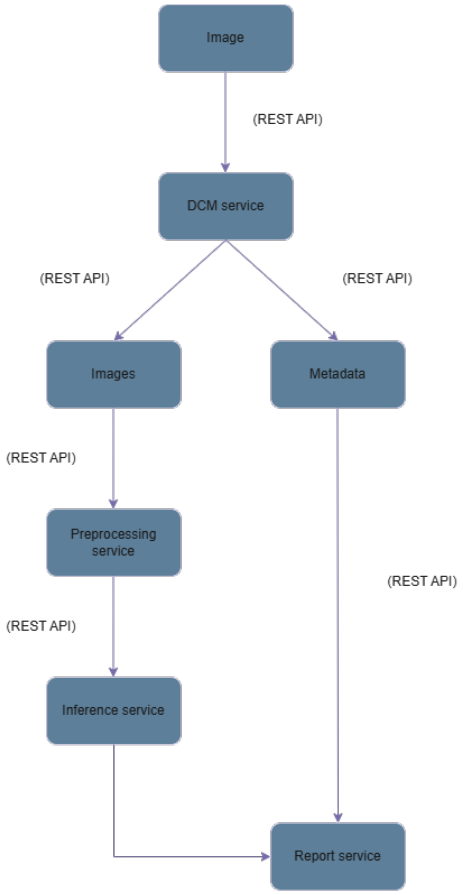


Figure 4.1: Diagram of the X-Ray Pneumonia Detection Application Architecture.

Chapter 5

Implementation

We have implemented a cloud-native application composed of 3 microservices to carry out the AI-assisted diagnosis of pneumonic patients. These microservices are individually containerized and the app relies on a pipeline (DCM microservice → Preprocessing microservice → Inference microservice → Report microservice) to produce the final result that the end user will retrieve.

In the first instance, a Docker container was defined. The container requires Python 3 and installs dependencies listed in the requirements file, which include libraries for image processing, cloud storage, and web APIs.

For the preprocessing service, we defined a class that loads and preprocesses images using a pipeline of resizing, grayscale conversion, blurring, histogram equalization, and normalization, saves processed images to a Google Cloud Storage bucket, and exposes this functionality via Flask REST API.

For the inference service, we defined a class that loads a pre-trained deep learning model from a specified path, processes input images received from the preprocessing service to generate predictions, and returns the inference results in a structured JSON format for further processing by the report service.

For the report service, we defined a class that aggregates the inference results, generates a comprehensive diagnostic report and allows the end user to retrieve the results.

The Dockerfile sets up the environment, exposes port 5000, and runs the main script when the container is initialized.

Chapter 6

Deployment

The deployment of our application on kubernetes was done by the following script:

```
docker build -t gcr.io/cloud-computing-project-2025/xray-app .
docker tag gcr.io/cloud-computing-project-2025/xray-app gcr.io/cloud-computing-project-2025/xray-app:v1
docker push gcr.io/cloud-computing-project-2025/xray-app
docker push gcr.io/cloud-computing-project-2025/xray-app:v1

gcloud container clusters create-auto xray-cluster --region europe-west4

gcloud container clusters get-credentials xray-cluster --region europe-west4

kubectl create serviceaccount xray-app-sa --dry-run=client -o yaml | kubectl apply -f -

kubectl apply -f https://raw.githubusercontent.com/kubernetes/ingress-nginx/controller-v1.6.4/deploy/static/provider/cloud/deploy.yaml

kubectl apply -f new_deployment.yaml

kubectl get pods
kubectl get deployments
kubectl get hpa
kubectl get pods -n ingress-nginx
kubectl get svc -n ingress-nginx
```

Chapter 7

Testing and Evaluation

IP: http://34.76.93.84
Endpoints tested:

- /GBD/GetMoviesData
- /GBD/CreatePlaylist
- /GBD/GetAllUsers
- /GBD/GetMovieID/MovieID
- /GBD/GetMoviesByActor/ActorID
- /GBD/GetMoviesByGenre/GenreID

7.1 Stress test

The stress test was performed with `https://locust.io` and evaluates the capacity of several endpoints of the target project in terms of requests per second (RPS).

- RPS: 500 - 700
- Failure Rate: 16%

7.2 Smoke test

The smoke test was performed with `https://www.postman.com` and assessed the main endpoints from the functional point of view and checked the main use cases.

- All tested endpoints responded without error codes and returned valid, expected values.
- In some cases the timeout test - set to 4000 ms - would fail. However, the maximum recorded response time was 7000 ms and the median of the failing tests remained below 5000 ms returns.

7.3 Takes and Feedback

The application functions correctly and can scale appropriately with the number of incoming requests. While the addition of bearer token authentication was a nice feature, we believe it was unnecessary for testing, as the initial tests resulted in a 100% error rate. This required contacting the target group to bypass the authentication barrier.

Currently, the error-free RPS limit is around 500, as higher values were observed to result in failed requests due to overload. In discussions with the target group, it was noted that although the database system is document-based, no query indexing was implemented. By introducing indexing, we believe the current RPS cap could be raised, and most - if not all - smoke test failures caused by timeouts would be resolved.

7.4 Corrections implemented after feedback from testing

After the feedback received from the target group we have separated each microservice in its own container, achieving independent scalability (each service can be scaled in an isolated way without necessarily impacting remaining microservices) and completed the SWAGGER API for each microservice, allowing us to carry out broader testing (unit and stress/smoke tests can now be done in individual containers).

However, we foresee a potential increase in operating costs as each container incurs an additional cost to memory/CPU.

Chapter 8

Cost Analysis

The cost of the project lies on kubernetes. Although the service has a free component, the required usage of pods makes the cost rise rampantly.

8.1 Estimated monthly costs:

1. Google Kubernetes Engine (GKE) Autopilot Cluster Cluster Management Fee: $0 (Autopilot mode, no management fee)

   Node/Pod Resources:

   Since we are considering 5 pods (via HPA), each with up to 2 CPUs and 15Gi RAM.

   vCPU: €0.096/hour RAM: €0.011/hour per GiB Max resources (5 pods):

   vCPU: 2 x 5 = 10 vCPU RAM: 15 x 5 = 75 GiB Monthly (730 hours):

   vCPU: 10 x €0.096 x 730 €700 RAM: 75 x €0.011 x 730 €600 Total (max scale): €1,300/month

2. Container Registry (Artifact Registry) Storage: $0.10/GB/month (for stored images)

8.2 FinOps Measures

- Only create GKE clusters and cloud resources when it's necessary to (re)train the model. After this, rely on automation scripts to accelerate deployment and deployment deletion.
- Always delete clusters, VMs, and cloud storage after training is complete to avoid extra costs for unnecessary resources. Integrate cleanup steps in your stop script to ensure all cloud resources (clusters, images, disks, IPs) are deleted after use.
- Run development, testing, and inference tasks on local hospital computers to eliminate unnecessary cloud costs.
- Set up GCP billing alerts to notify you if any resources remain active or if costs exceed expected thresholds.

Chapter 9

Conclusions

This project successfully demonstrates the design and implementation of a cloud-native AI tool to help doctors detect pneumonia in chest X-rays. The latest improvement, that focuses on containerizing microservices, allows the app to be both more scalable and easier to maintain/test. However, due to the limited budget we were working with, and in order to keep costs low, we thought best to make some concessions and develop an app that currently handles a few dozens of users/images concurrently, and is predicted to scale accordingly with the addition of more resources.

As we imagine this app being implemented in a real-world setting, we envision that the development, testing and diagnosis would happen locally. This allows us to only pay for cloud resources when absolutely necessary, keeping things efficient without sacrificing performance.

The containerized approach means each part of the system can scale independently, so resources aren't wasted. We also automated deployments to make the whole process smoother and avoid surprise costs.